SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA1017	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Nithiyasri S	**Reg. No.:**	192572082
Date:	13-08-2026	**Duration:**	60 minutes

Code of Conduct

I Nithiyasri S (192572082) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Nithiyasri S

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. **Problem Analysis**
   - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
2. **Project Structure Design**
   - Design and explain the folder structure of your project repository.
   - Justify the purpose of each major folder and file included in the repository.
3. **Git Repository Initialization**
   - Create a Git repository for the project.
   - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
4. **Git Branching Strategy**
   - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
   - Justify why the selected branching model is appropriate for the assigned project.
5. **Version Control Workflow**
   - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
   - Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
    - Present the commit history of the project.
    - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
    - Explain why Docker is suitable for the assigned project.
    - Identify the application components that require containerization.
8. **Dockerfile Development**
    - Design and explain the Dockerfile required to containerize the application.
    - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
    - Develop a Docker Compose configuration for the project.
    - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
    - Demonstrate the execution of the application using Docker containers.
    - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
    - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
    - Identify the challenges encountered during implementation.
    - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.

Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

1. Problem Analysis

1.1 Problem Statement

Students and job seekers frequently struggle to identify career paths that genuinely match their skills, aptitude, and interests because conventional career counselling is generic, infrequent, and disconnected from real-time labour-market signals. Educational institutions and training providers therefore need a data-driven system that objectively evaluates a learner's technical and soft skills, benchmarks those skills against live industry requirements, and translates the gap into a concrete, personalised action plan covering career pathways, certification programs, and job opportunities. The AI-Powered Smart Vocational Skill Assessment and Career Recommendation Platform is designed to close this education-to-employment gap using AI-driven assessment and recommendation techniques.

1.2 Project Objectives

- Assess a learner's technical and soft skills through structured, adaptive online tests.
- Use AI models to analyse aptitude, learning preferences, and behavioural patterns.
- Recommend suitable career paths ranked by fit score and market demand.
- Match individual competencies against current industry skill requirements.
- Suggest relevant training courses and certification programs to close identified gaps.
- Generate personalised, downloadable career reports for each learner.
- Continuously monitor learning progress and re-evaluate recommendations over time.
- Improve overall employability and workforce readiness of the learner base.

1.3 Functional Requirements

- User registration, authentication, and profile management for learners, institutions, and administrators.
- Configurable skill-assessment engine supporting MCQ, coding, and psychometric question types.
- AI/ML recommendation service that scores aptitude and maps it to career clusters.
- Industry skill-demand data ingestion (via APIs or curated datasets) for gap analysis.
- Career report generation module producing a downloadable PDF/summary per learner.
- Dashboard for progress monitoring, retesting, and notification of new recommendations.
- Admin console for managing question banks, career taxonomies, and analytics.

1.4 Expected Outcomes

- Personalised, evidence-based career recommendations for every learner.
- Improved student employability and better skill-to-job matching.
- Increased career awareness and enhanced workforce readiness.
- Data-driven career planning leading to higher placement success and lifelong learning.

2. Project Structure Design

The application follows a modular, service-oriented repository layout that separates the frontend, backend APIs, the AI/ML recommendation engine, infrastructure configuration, and documentation. This separation keeps each concern independently testable, containerizable, and deployable while still living inside a single, version-controlled monorepo for ease of coordinated releases.

```
ai-career-platform/
├── frontend/                # React.js single-page application
│   ├── public/
│   ├── src/
│   │   ├── components/      # Reusable UI components
│   │   ├── pages/           # Assessment, Dashboard, Report pages
│   │   ├── services/        # Axios API clients
│   │   └── App.js
│   ├── package.json
│   └── Dockerfile
├── backend/                  # Node.js / Express REST API
│   ├── src/
│   │   ├── routes/          # auth, assessment, career, admin routes
│   │   ├── controllers/
│   │   ├── models/          # Sequelize / Mongoose schemas
│   │   ├── middleware/      # JWT auth, validation, error handling
│   │   └── server.js
│   ├── package.json
│   └── Dockerfile
├── ml-service/                # Python FastAPI AI/ML microservice
│   ├── app/
│   │   ├── models/          # Trained aptitude & recommendation models
│   │   ├── routers/         # /predict, /recommend endpoints
│   │   └── main.py
│   ├── requirements.txt
│   └── Dockerfile
├── database/
│   └── init.sql             # Schema & seed data for PostgreSQL
├── docs/                      # Architecture & workflow diagrams
├── docker-compose.yml
├── .env.example
├── .gitignore
├── .github/workflows/ci.yml   # CI pipeline definition
├── LICENSE
└── README.md
```

2.1 Purpose of Major Folders and Files

Folder / File	Purpose
frontend/	Contains the React SPA that learners and administrators use to take assessments and view recommendations.
backend/	Node.js/Express service exposing REST APIs for authentication, assessment delivery, and report retrieval.
ml-service/	Python FastAPI microservice hosting the trained aptitude-analysis and career-recommendation models.
database/init.sql	Defines the initial PostgreSQL schema (users, skills, assessments, careers) and seed reference data.
docker-compose.yml	Orchestrates all containers (frontend, backend, ml-service, database, cache) as one multi-container application.
.env.example	Documents required environment variables without exposing actual secrets in version control.
.gitignore	Excludes node_modules, virtual environments, build artefacts, and secrets from the repository.
.github/workflows/ci.yml	Defines the automated build/test pipeline triggered on every push and pull request.
README.md	Onboarding documentation: setup instructions, architecture summary, and contribution guidelines.

3. Git Repository Initialization

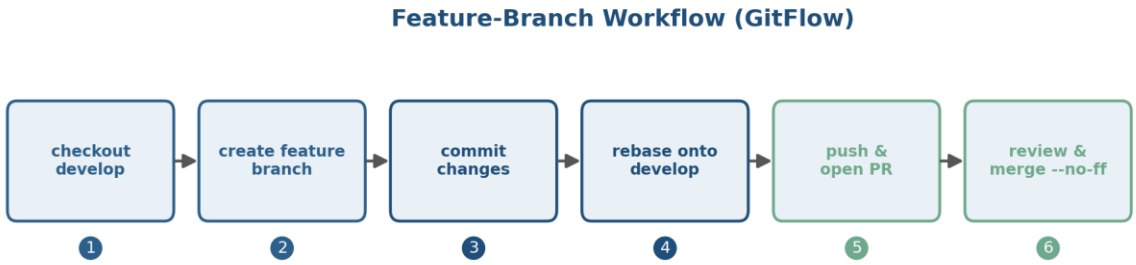
The project repository was initialised locally and then linked to a remote hosting service (GitHub) to enable collaborative development, backup, and integration with CI/CD pipelines.

3.1 Initialization Steps

```
# 1. Create the project folder and initialise Git
mkdir ai-career-platform && cd ai-career-platform
git init

# 2. Configure author identity for this repository
git config user.name "Student Name"
git config user.email "student@saveetha.ac.in"

# 3. Add a .gitignore and initial project files
touch .gitignore README.md
git add .
git commit -m "chore: initial commit - project scaffold"

# 4. Create and link the remote repository
git remote add origin https://github.com/<username>/ai-career-platform.git
git branch -M main
git push -u origin main
```

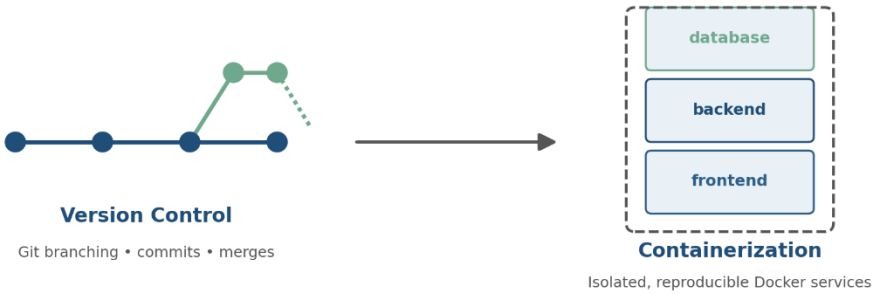
3.2 Purpose of Essential Git Files

File	Purpose
.git/	Hidden directory holding the entire object database, refs, and history — created by git init.
.gitignore	Prevents build artefacts (node_modules, __pycache__, .env, dist/) from being tracked.
README.md	Describes the project, setup steps, and usage for new contributors.
LICENSE	Declares the terms under which the source code may be reused.
.github/workflows/ci.yml	Enables GitHub Actions to automatically build and test each commit.



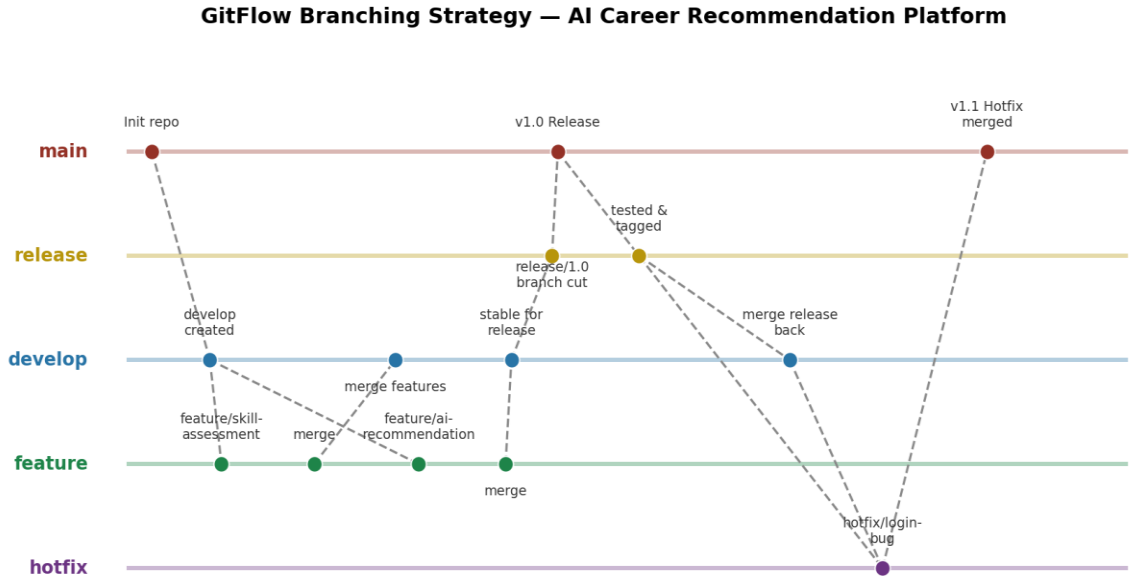
4. Git Branching Strategy

The project adopts the GitFlow branching model because it cleanly separates ongoing feature development from stabilisation and production releases — an important property for a multi-service application (frontend, backend, ML microservice) built and graded incrementally over the assignment timeline.



4.1 Branch Types

Branch	Purpose
main	Always reflects the latest production-ready release; every commit here is tagged with a version.
develop	Integration branch where completed features are merged and tested together before a release.
feature/*	Short-lived branches for individual features, e.g. feature/skill-assessment, feature/ai-recommendation.
release/*	Cut from develop when preparing a release; used for final QA, bug fixing, and documentation.
hotfix/*	Branched from main to urgently patch a production defect without pulling in unfinished develop work.



4.2 Justification

- Parallel feature work: the frontend, backend, and ML-service teams can develop skill-assessment, recommendation, and reporting features on isolated feature branches without blocking one another.
- Stable main branch: main only ever contains tested, deployable code, which is essential since the Docker images are built directly from it.
- Controlled releases: the release/* branch allows final regression testing and documentation updates before a version is tagged and containers are rebuilt.

5. Version Control Workflow

The following sequence demonstrates a typical feature-development cycle, including branch creation, commits, synchronisation, merging, and conflict resolution.

```
# Create a feature branch from develop
git checkout develop
git pull origin develop
git checkout -b feature/ai-recommendation

# Make meaningful, incremental commits
git add ml-service/app/routers/recommend.py
git commit -m "feat(ml-service): add /recommend endpoint for career scoring"
git add ml-service/app/models/recommender.py
git commit -m "feat(ml-service): integrate trained RandomForest recommendation model"

# Keep the feature branch up to date with develop
git fetch origin
git rebase origin/develop

# Push the feature branch and open a Pull Request
git push -u origin feature/ai-recommendation

# After PR review and approval, merge into develop
git checkout develop
git merge --no-ff feature/ai-recommendation
git push origin develop
```

5.1 Conflict Resolution Example

During integration, a conflict arose in docker-compose.yml because two feature branches independently added new environment variables to the ml-service definition. The conflict was resolved by manually merging both sets of variables and re-testing the container build:

```
git merge feature/ai-recommendation
# CONFLICT (content): Merge conflict in docker-compose.yml

# Open the file, keep both variable blocks under ml-service.environment,
# remove the conflict markers <<<<<<<, =======, >>>>>>>, then:
git add docker-compose.yml
git commit -m "merge: resolve docker-compose env conflict between feature branches"
git push origin develop
```

5.2 Purpose of Each Operation

Operation	Purpose
git checkout -b	Creates and switches to a new isolated branch for a feature.
git commit	Records a logical, reviewable snapshot of change with a descriptive message.
git rebase	Replays local commits on top of the latest develop to keep history linear and avoid drift.
git push -u	Publishes the branch to the remote and sets tracking for future pushes.
git merge --no-ff	Merges the feature into develop while preserving a merge commit for traceability.
conflict resolution	Manually reconciles competing edits to the same file/lines before completing a merge.

6. Commit History Analysis

A representative commit history for the project, viewed with git log --oneline --graph, is shown below.

```
* 9f3a1c2 (HEAD -> main) release: tag v1.1.0 - hotfix login token expiry
* 7b2e410 hotfix: fix JWT expiry check causing false session timeouts
* 6d1c9aa merge: release/1.0 into main
* 5a8f221 (tag: v1.0.0) chore: bump version to 1.0.0
* 4c77e10 docs: update README with docker-compose usage
* 3f902bb merge: develop into release/1.0
* 2e11caa merge: resolve docker-compose env conflict between feature branches
* 1a56de3 feat(ml-service): integrate trained RandomForest recommendation model
* 0d44f19 feat(ml-service): add /recommend endpoint for career scoring
* c39ab02 feat(backend): add assessment scoring controller and routes
* b21f8e0 feat(frontend): build skill-assessment quiz UI
* a10ee55 chore: initial commit - project scaffold
```

6.1 Good Version-Control Practices Observed

- Conventional commit prefixes (feat, fix, chore, docs, merge) make the history self-documenting and let tools auto-generate changelogs.
- Each commit is scoped to a single logical change (one feature, one fix), which keeps diffs small and reviewable.
- Commit subject lines are written in the imperative mood and stay under ~50 characters for readability in git log.
- Release points are tagged (v1.0.0, v1.1.0), giving reproducible references for Docker image builds.
- Merge commits are preserved (--no-ff) so the graph shows exactly when and where features and hotfixes entered develop and main.

Together these practices make it possible to trace any production issue back to the exact commit and branch that introduced it, which directly supports the project's maintainability and audit requirements.

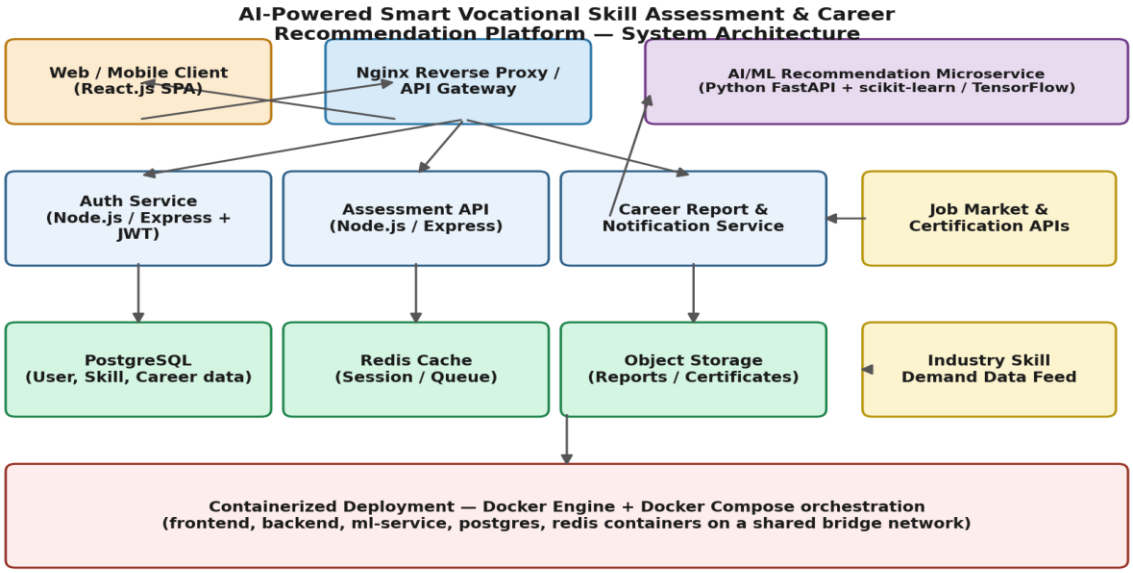
7. Docker Environment Design

7.1 Why Docker is Suitable

- The platform is composed of three independently built services (React frontend, Node.js backend, Python ML microservice) plus PostgreSQL and Redis — Docker lets each run in an isolated, reproducible environment with its own dependencies.
- Containerization removes 'works on my machine' inconsistencies between student development laptops, CI runners, and any deployment target.
- Docker Compose allows the entire multi-service stack to be started with a single command, which is essential for grading, demos, and onboarding.
- Images can be version-tagged and pushed to a registry, enabling consistent, rollback-capable deployments.
- Horizontal scaling (e.g. running multiple ml-service replicas under load) is straightforward once the service is containerized.

7.2 Components Requiring Containerization

Component	Containerization Need
frontend	Needs Node.js build tooling at build-time and a lightweight static server (Nginx) at runtime.
backend	Needs a Node.js runtime with its own package set, isolated from the frontend and ML service.
ml-service	Needs Python plus heavy ML libraries (scikit-learn/TensorFlow) — best kept isolated from Node services.
postgres	Needs a persistent, version-pinned database engine with its own data volume.
redis	Needs an isolated in-memory store for sessions/queues, independent of the application containers.



8. Dockerfile Development

8.1 Backend Dockerfile (Node.js / Express)

```
# backend/Dockerfile
FROM node:20-alpine AS base
WORKDIR /usr/src/app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

ENV NODE_ENV=production
EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=5s \
  CMD wget -qO- http://localhost:5000/health || exit 1

CMD ["node", "src/server.js"]
```

8.2 ML Service Dockerfile (Python / FastAPI)

```
# ml-service/Dockerfile
FROM python:3.11-slim
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

8.3 Frontend Dockerfile (React, multi-stage build)

```
# frontend/Dockerfile
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:1.27-alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

8.4 Instruction Reference

Instruction	Purpose
FROM	Selects the base image (Node, Python, or Nginx) the container is built from.
WORKDIR	Sets the working directory inside the container for all subsequent instructions.
COPY	Copies source files or build artefacts from the host (or a previous build stage) into the image.
RUN	Executes a command at build time, e.g. installing dependencies.
ENV	Sets environment variables available to the running container.
EXPOSE	Documents the port the container listens on.
HEALTHCHECK	Lets Docker periodically verify the service is responsive.
CMD	Specifies the default command executed when the container starts.
Multi-stage build (AS build)	Keeps the final frontend image small by discarding Node build tooling and shipping only static assets.

9. Docker Compose Design

```
# docker-compose.yml
version: "3.9"

services:
  frontend:
    build: ./frontend
    ports:
      - "3000:80"
    depends_on:
      - backend
    networks:
      - career-net

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      - DATABASE_URL=postgresql://app_user:app_pass@postgres:5432/career_db
      - REDIS_URL=redis://redis:6379
      - ML_SERVICE_URL=http://ml-service:8000
      - JWT_SECRET=${JWT_SECRET}
    depends_on:
      - postgres
      - redis
      - ml-service
    networks:
      - career-net
```

```
  ml-service:
    build: ./ml-service
    ports:
      - "8000:8000"
    volumes:
      - ml-models:/app/app/models
    networks:
      - career-net

  postgres:
    image: postgres:16-alpine
    environment:
      - POSTGRES_USER=app_user
      - POSTGRES_PASSWORD=app_pass
      - POSTGRES_DB=career_db
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql
    networks:
      - career-net

  redis:
    image: redis:7-alpine
    networks:
      - career-net

volumes:
  pgdata:
  ml-models:

networks:
  career-net:
    driver: bridge
```

9.1 Design Explanation

Aspect	Explanation
Services	Five services model the frontend, backend API, ML microservice, and their PostgreSQL/Redis data stores as separate, independently scalable units.
Networking	All services share the career-net bridge network, so containers resolve each other by service name (e.g. postgres, ml-service) instead of hard-coded IPs.
Volumes	pgdata persists database contents across container restarts; ml-models persists trained model artefacts so they survive container rebuilds.
Environment variables	Connection strings and secrets (DATABASE_URL, JWT_SECRET) are injected at runtime, keeping credentials out of source code.
Dependencies	depends_on ensures the backend only starts after postgres, redis, and ml-service are scheduled, preventing early connection failures.

10. Container Deployment and Testing

10.1 Building and Running the Stack

```
# Build all images and start the stack in the background
docker compose build
docker compose up -d

# Verify all containers are running and healthy
docker compose ps

NAME                        STATUS
career-frontend-1          Up 2 minutes
career-backend-1           Up 2 minutes (healthy)
career-ml-service-1        Up 2 minutes
career-postgres-1          Up 2 minutes
career-redis-1             Up 2 minutes
```

10.2 Functional Testing Procedure

- Health check: curl http://localhost:5000/health returned HTTP 200 confirming the backend API is reachable.
- ML endpoint test: a sample skill profile POSTed to http://localhost:8000/recommend returned a ranked list of career matches with confidence scores.
- End-to-end test: a learner account was registered through the frontend at http://localhost:3000, completed a sample assessment, and received a generated career report, confirming that frontend → backend → ml-service → database all interoperate correctly inside the containers.
- Log verification: docker compose logs -f backend was used to confirm requests were being received and processed without errors during the above tests.
- Persistence test: docker compose down followed by docker compose up -d confirmed learner data survived the restart because it was stored in the pgdata volume.

10.3 Evidence of Successful Execution

All five containers reported a running/healthy state, the assessment workflow completed end-to-end through the browser, and the /recommend API returned valid JSON career recommendations, confirming the containerized application functions as designed. (Insert actual terminal and browser screenshots here when submitting.)

11. Benefits of Git and Docker

Dimension	Benefit to this Project
Collaboration	GitFlow branches let frontend, backend, and ML contributors work in parallel without blocking each other, then integrate through reviewed pull requests.
Version management	Every change to the assessment engine, API, or ML model is traceable to a commit, author, and timestamp, supporting accountability and rollback.
Portability	Docker images encapsulate each service's runtime, so the platform runs identically on a student laptop, a CI runner, or a cloud VM.
Deployment	docker compose up reproduces the full multi-service environment in minutes, drastically shortening setup and demo time.
Scalability	Stateless services (backend, ml-service) can be scaled to multiple replicas behind the API gateway as learner load grows.
Maintainability	Small, well-tagged commits and isolated containers make it easy to locate, fix, and redeploy a single faulty component without touching the rest of the system.

12. Challenges and Improvements

12.1 Challenges Encountered

- Merge conflicts in docker-compose.yml when multiple feature branches added environment variables concurrently.
- Coordinating API contracts between the Node.js backend and the Python ML microservice required careful versioned schema documentation to avoid breaking changes.
- Initial container start-up ordering issues, where the backend attempted to connect to PostgreSQL before it was ready, required depends_on plus application-level retry logic.
- Managing trained ML model artefacts (large binary files) inside Git required excluding them via .gitignore and instead persisting them through a Docker volume.

12.2 Suggested Improvements and Future Enhancements

- Adopt Docker health checks with `condition: service_healthy` in Compose (or a full orchestrator like Kubernetes) to guarantee strict start-up ordering.
- Introduce Git commit hooks (Husky / pre-commit) to enforce conventional commit messages and run linting before code is pushed.
- Extend the CI pipeline to automatically build and test all Docker images on every pull request before merging into develop.
- Adopt a model registry (e.g. MLflow) so ML model versions are tracked independently of application code releases.
- Add centralized logging and monitoring (ELK stack / Prometheus-Grafana) across containers for production observability.